Création progressive d'une première application Flutter

Développement Mobile avec Dart & Flutter

Pr. Abdelfattah Abassi

Faculté des Sciences et Techniques de Settat
Université Hassan I

Laboratoire LAVETE
Département de Physique Appliquée

Année universitaire 2026–2027

Objectifs pédagogiques

À la fin de cette partie, l'étudiant sera capable de :

- créer un projet Flutter ;
- comprendre la structure d'un projet ;
- identifier le rôle de `main.dart` ;
- comprendre `main()` et `runApp()` ;
- utiliser `MaterialApp` et `Scaffold` ;
- construire progressivement une interface ;
- utiliser `AppBar`, `Center`, `Container` et `Text` ;
- ajouter un bouton ;
- gérer une première interaction utilisateur ;
- utiliser le Hot Reload ;
- diagnostiquer les erreurs élémentaires.

Plan de la partie

1. 1. Création du projet
2. 2. Structure du projet
3. 3. main.dart et main()
4. 4. runApp()
5. 5. MaterialApp et Scaffold
6. 6. Hello World
7. 7. AppBar
8. 8. Center
9. 9. Container

Créer une première application

Nous allons construire une application Flutter progressivement.

Point de départ

Notre projet s'appellera :

`hello_flutter`

Nous partirons d'une application minimale puis nous ajouterons progressivement :

- une structure Material ;
- une barre d'application ;
- du contenu ;
- une mise en forme ;
- un bouton ;
- une interaction.

Commande : flutter create

Dans un terminal :

```
1          flutter create hello_flutter
```

Cette commande demande à Flutter de générer automatiquement un nouveau projet.

Résultat

Un dossier nommé `hello_flutter` est créé.

Que fait flutter create ?

La commande :

```
flutter create hello_flutter
```

génère notamment :

- le code Dart initial ;
- la configuration Android ;
- la configuration iOS ;
- la configuration Web ;
- les fichiers de test ;
- le fichier `pubspec.yaml`.

Idée importante

Flutter prépare une structure multi-plateforme à partir d'une seule commande.

Entrer dans le projet

Après la création :

```
1          cd hello_flutter
```

cd signifie :

Change Directory

La commande permet de se déplacer dans le dossier du projet.

À retenir

Les commandes Flutter doivent généralement être exécutées depuis le dossier du projet.

Lancer l'application

Pour lancer l'application :

```
1          flutter run
```

Flutter :

1. recherche un appareil disponible ;
2. compile l'application ;
3. installe l'application ;
4. lance l'application.

Vérifier les appareils disponibles

Avant `flutter run`, on peut utiliser :

```
flutter devices
```

Flutter peut détecter par exemple :

- un smartphone Android ;
- un émulateur Android ;
- Chrome ;
- un appareil iOS sur macOS ;
- d'autres cibles compatibles.

Vérifier l'installation de Flutter

Commande :

```
1          flutter doctor
```

Cette commande analyse l'environnement de développement.

Elle permet notamment d'identifier :

- Flutter ;
- Android SDK ;
- Android Studio ;
- les appareils ;
- les outils nécessaires.

Interpréter flutter doctor

On peut rencontrer :

Situation normale

Des éléments apparaissent comme correctement configurés.

Attention

Une croix ou un avertissement indique qu'un composant nécessite éventuellement une configuration.

Conseil

Toujours exécuter `flutter doctor` lorsqu'un problème d'environnement apparaît.

Structure générale

Un projet Flutter contient plusieurs éléments :

- `lib/`
- `main.dart`
- `test/`
- `android/`
- `ios/`
- `web/`
- `assets/`
- `pubspec.yaml`

Le dossier lib/

Le dossier :

`lib/`

contient principalement le code Dart de l'application.

Principe

Le développeur travaille principalement dans `lib/`.

On peut y organiser :

- les écrans ;
- les widgets ;
- les modèles ;
- les services ;
- les fichiers utilitaires.

main.dart

Le fichier principal est généralement :

`lib/main.dart`

Il contient le point d'entrée initial de l'application.

Question

Que doit faire un programme lorsqu'il démarre ?

Il doit savoir où commencer son exécution.

Le dossier test/

Le dossier :

`test/`

est destiné aux tests.

On peut y écrire :

- des tests unitaires ;
- des tests de widgets ;
- des tests permettant de vérifier le comportement de l'application.

Bonne pratique

Les tests permettent de détecter rapidement les régressions.

Le dossier android/

Le dossier :

`android/`

contient la partie spécifique Android du projet Flutter.

Il permet notamment de gérer :

- la configuration Android ;
- les paramètres de compilation ;
- certaines intégrations natives.

Le dossier ios/

Le dossier :

`ios/`

contient la partie spécifique à iOS.

Il devient particulièrement important lorsque l'application utilise des fonctionnalités natives propres à Apple.

Le dossier web/

Le dossier :

`web/`

contient les éléments nécessaires à la cible Web.

Flutter permet ainsi de développer une application pouvant être exécutée :

- sur Android ;
- sur iOS ;
- sur le Web ;
- et sur d'autres plateformes compatibles.

Le dossier assets/

Un dossier `assets/` peut être créé pour stocker :

- images ;
- icônes ;
- fichiers JSON ;
- autres ressources.

Exemple :

```
1           assets/
2           images/
3           data/
```

Les ressources doivent ensuite être déclarées dans `pubspec.yaml`.

pubspec.yaml

Le fichier :

`pubspec.yaml`

est un fichier de configuration essentiel.

Il permet notamment de déclarer :

- les dépendances ;
- les ressources ;
- certaines informations du projet ;
- les contraintes de version.

Exemple de pubspec.yaml

Extrait simplifié :

```yaml
name: hello_flutter
description: Une premiere application Flutter

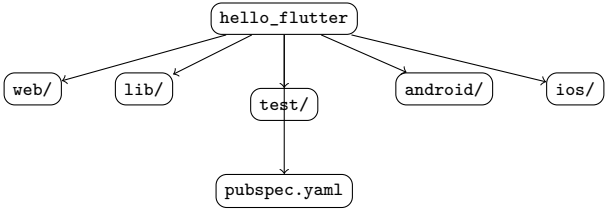
environment:
sdk: ">=3.0.0 <4.0.0"

dependencies:
flutter:
sdk: flutter
```

Attention

L'indentation YAML est significative.

Vue conceptuelle du projet



À retenir : structure

- `lib/` : code Dart principal ;
- `main.dart` : point d'entrée ;
- `test/` : tests ;
- `android/` : intégration Android ;
- `ios/` : intégration iOS ;
- `web/` : cible Web ;
- `assets/` : ressources ;
- `pubspec.yaml` : configuration et dépendances.

Le point d'entrée

Dans Dart, l'exécution commence généralement par :

```
main()
```

Cette fonction constitue le point d'entrée du programme.

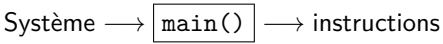
Première fonction main()

```dart
void main() {
    print('Bonjour Flutter');
}
```

Analyse :

- `void` : la fonction ne retourne pas de valeur ;
- `main` : nom spécial du point d'entrée ;
- `{}` : corps de la fonction ;
- `print()` : affiche un message.

Fonction main : schéma



L'application commence son exécution à partir de cette fonction.

main() et Flutter

Une application Flutter commence généralement ainsi :

```
1  void main() {
2      runApp(const MyApp());
3  }
```

Deux notions apparaissent :

1. `main()` ;
2. `runApp()`.

Qu'est-ce que runApp() ?

`runApp()` indique à Flutter quel Widget doit être placé comme racine de l'interface.

Idée fondamentale

```
runApp(widget)
```

Flutter commence alors à construire l'interface à partir de ce Widget.

Premier runApp()

```dart
void main() {
    runApp(
    const Text('Hello Flutter'),
    );
}
```

Ici, le Widget racine est simplement un `Text`.

Pourquoi un Widget ?

Flutter construit l'interface sous forme d'un arbre de Widgets.

Exemple :

$$\text{Application} \longrightarrow \text{Widget racine} \longrightarrow \text{Widgets enfants}$$

Principe fondamental

Dans Flutter, l'interface est construite en composant des Widgets.

MyApp comme Widget racine

Nous allons créer notre propre Widget :

```
class MyApp extends StatelessWidget {
    const MyApp({super.key});

    @override
    Widget build(BuildContext context) {
        return const Text('Hello Flutter');
    }
}
```

Décomposition de MyApp

- `class MyApp` : création d'une classe ;
- `extends StatelessWidget` : Widget sans état mutable ;
- constructeur `const` ;
- `build()` : construit l'interface ;
- `BuildContext` : contexte du Widget ;
- `return` : Widget retourné.

MaterialApp

`MaterialApp` fournit une base pour une application utilisant Material Design.

Il permet notamment de gérer :

- le thème ;
- les routes ;
- le titre ;
- certaines configurations globales.

Premier MaterialApp

```
class MyApp extends StatelessWidget {
    const MyApp({super.key});

    @override
    Widget build(BuildContext context) {
        return MaterialApp(
        home: Text('Hello Flutter'),
        );
    }
}
```

Pourquoi MaterialApp ?

Une application réelle a besoin d'une structure plus complète.

`MaterialApp` fournit un environnement adapté aux applications Material.

On pourra ensuite utiliser :

- `Scaffold` ;
- `AppBar` ;
- boutons ;
- thèmes ;
- navigation.

Scaffold

`Scaffold` fournit une structure visuelle classique.

Il permet notamment de définir :

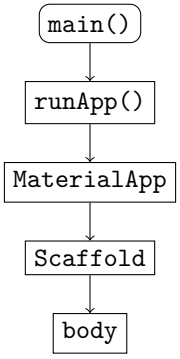
- `appBar` ;
- `body` ;
- `floatingActionButton` ;
- `drawer` ;
- `bottomNavigationBar`.

MaterialApp + Scaffold

```
1           return MaterialApp(
2           home: Scaffold(
3           body: Text('Hello Flutter'),
4           ),
5           );
```

Nous obtenons maintenant une structure d'application plus réaliste.

Architecture minimale



Version 1 : Hello World

Objectif :

Afficher simplement :

Hello Flutter

Nous allons construire cette interface étape par étape.

Hello World : code complet

```dart
import 'package:flutter/material.dart';

void main() {
    runApp(const MyApp());
}

class MyApp extends StatelessWidget {
    const MyApp({super.key});

    @override
    Widget build(BuildContext context) {
        return const MaterialApp(
        home: Scaffold(
        body: Text('Hello Flutter'),
        ),
        );
    }
}
```

Analyse du code

Le programme contient :

1. un import Flutter ;
2. `main()` ;
3. `runApp()` ;
4. `MyApp` ;
5. `MaterialApp` ;
6. `Scaffold` ;
7. `Text`.

L'import Flutter

```dart
import 'package:flutter/material.dart';
```

Cet import donne accès à de nombreux Widgets Material.

Par exemple :

- `MaterialApp`
- `Scaffold`
- `Text`
- `AppBar`
- `ElevatedButton`

Pourquoi const ?

On rencontre souvent :

```
1          const MyApp()
```

et :

```
1          const Text('Hello Flutter')
```

`const` indique qu'une valeur peut être créée comme constante à la compilation.

Bonne pratique

Utiliser `const` lorsque Flutter et Dart le permettent.

Le rôle de Scaffold

Dans notre application :

```
1           Scaffold(
2           body: Text('Hello Flutter'),
3           )
```

`Scaffold` fournit la structure visuelle.

Le contenu principal est placé dans :

`body`

Version 2 : ajouter une AppBar

Nous voulons maintenant obtenir :

Une barre supérieure + un contenu

Nous utiliserons :

`AppBar`

Ajouter AppBar

```
1           Scaffold(
2           appBar: AppBar(
3           title: const Text('Ma première application'),
4           ),
5           body: const Text('Hello Flutter'),
6           )
```

Comprendre AppBar

`AppBar` représente généralement la barre supérieure.

On peut y placer :

- un titre ;
- une icône de navigation ;
- des actions ;
- des boutons.

AppBar : title

```
1           appBar: AppBar(
2           title: const Text('Accueil'),
3           ),
```

Le paramètre :

`title`

reçoit un Widget.

AppBar avec actions

```
appBar: AppBar(
title: const Text('Accueil'),
actions: [
IconButton(
onPressed: () {},
icon: const Icon(Icons.settings),
),
],
),
```

À retenir : AppBar

- `AppBar` est placé dans `Scaffold`.
- Le titre est généralement un Widget.
- `actions` permet d'ajouter des actions.
- Les icônes sont disponibles via `Icons`.

Version 3 : centrer le texte

Notre texte est actuellement placé directement dans `body`.

Nous voulons le placer au centre de l'écran.

Nous utiliserons :

```
Center
```

Utiliser Center

```
1           body: const Center(
2           child: Text('Hello Flutter'),
3           ),
```

`Center` possède un enfant :

`child`

Concept parent / enfant

Dans Flutter :

`Center`

est le parent de :

`Text`

On peut représenter :

Center $\rightarrow$ Text

Center avec plusieurs Widgets

`Center` n'accepte qu'un seul `child`.

Pour plusieurs éléments, on pourra utiliser :

```
1           Center(
2           child: Column(
3           children: [
4           Text('Bonjour'),
5           Text('Flutter'),
6           ],
7           ),
8           )
```

Version 4 : Container

Nous allons maintenant contrôler l'apparence d'un élément.

Le Widget :

`Container`

permet notamment de gérer :

- dimensions ;
- espace intérieur ;
- espace extérieur ;
- alignement ;
- décoration.

Container simple

```
1           Container(
2           child: const Text('Hello Flutter'),
3           )
```

Le paramètre `child` contient le Widget enfant.

Container : width

```
1           Container(
2           width: 250,
3           child: const Text('Hello Flutter'),
4           )
```

`width` définit la largeur souhaitée.

Container : height

```
1           Container(
2           width: 250,
3           height: 100,
4           child: const Text('Hello Flutter'),
5           )
```

Nous contrôlons maintenant :

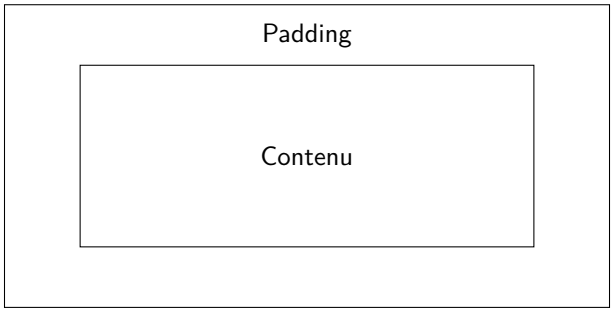
- largeur ;
- hauteur.

Container : padding

```
1           Container(
2           padding: const EdgeInsets.all(20),
3           child: const Text('Hello Flutter'),
4           )
```

Le padding représente l'espace intérieur.

Padding : représentation



Container : margin

```dart
Container(
margin: const EdgeInsets.all(20),
child: const Text('Hello Flutter'),
)
```

Le `margin` représente l'espace extérieur autour du conteneur.